

ONEDRAFT

CAD-AI — Aria Powered

Observability, Telemetry & System Health

Version 1.0

Status: Implementation Architecture Specification

System: OneDraft CAD-AI Platform

Primary Intelligence: Aria Powered

Scope: Platform Observability, Telemetry, Diagnostics, Monitoring and System Health

Dependencies: OneDraft Stacks 1.0–29.0

Architectural Position: Operational Systems Layer

1. PURPOSE

The OneDraft platform has now established its foundational project model, computational geometry system, regulatory and compliance engine, validation framework, documentation engine, review and acceptance workflow, runtime orchestration, asynchronous job system, artifact storage, integration/event architecture, and identity/security boundary.

The next requirement is the ability to continuously determine whether those systems are operating correctly.

OneDraft therefore requires a dedicated observability architecture.

The Observability, Telemetry & System Health subsystem provides the instrumentation necessary to determine:

what the system is doing

what the system has done

whether the system is functioning correctly

where a failure occurred

which project or operation was affected

which version of the model was involved

which service performed the operation

how long the operation required

whether downstream artifacts remain trustworthy

whether the system requires intervention

Observability becomes part of the OneDraft operational system of record.

2. OBSERVABILITY PRINCIPLE

OneDraft shall not depend upon individual service logs as its sole mechanism of system diagnosis.

The platform shall produce structured telemetry across:

```
CLIENT
↓
API GATEWAY
↓
IDENTITY / AUTHORIZATION
↓
DOMAIN SERVICES
↓
ARIA
↓
COMMAND SYSTEM
↓
GEOMETRY ENGINE
↓
COMPLIANCE ENGINE
↓
VALIDATION ENGINE
↓
DOCUMENTATION ENGINE
↓
ARTIFACT STORAGE
↓
JOB / WORKER SYSTEM
↓
EVENT BUS
↓
DATABASE
```

Each major operation must be traceable across the participating services.

The objective is not merely to record errors.

The objective is to establish **causal operational visibility**.

3. THREE PRIMARY TELEMETRY SIGNALS

OneDraft observability shall initially use three primary telemetry classes:

```
LOGS
METRICS
TRACES
```

These signals are complementary.

Logs explain:

what happened

Metrics explain:

how often and how much

Traces explain:

where the operation travelled through the system

4. FOURTH SIGNAL — SYSTEM EVENTS

Operational telemetry shall remain distinct from domain events.

Domain events describe changes to the OneDraft computational system.

Operational telemetry describes the behaviour of the software executing those changes.

For example:

DOMAIN EVENT
REVISION_CREATED

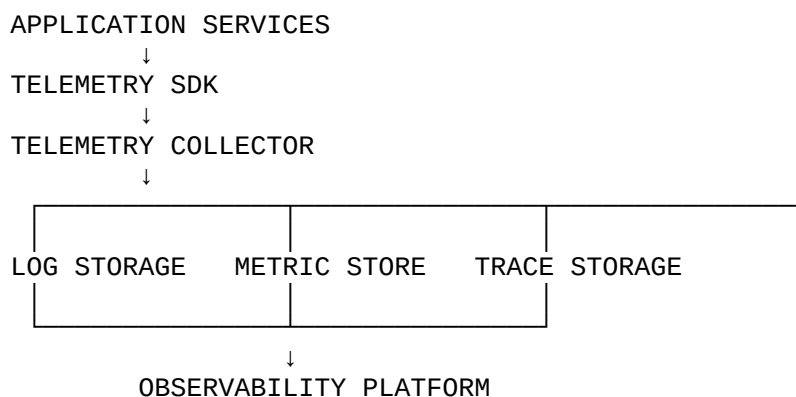
is distinct from:

OPERATIONAL EVENT
revision_service.create_revision.duration = 182ms

This distinction prevents operational diagnostics from contaminating the project domain model.

5. OBSERVABILITY ARCHITECTURE

The initial architecture shall be:



The application layer should emit telemetry through standardized interfaces rather than directly coupling domain services to a particular observability vendor.

6. TELEMETRY ABSTRACTION

The OneDraft application shall expose an internal telemetry abstraction.

Conceptually:

```
Telemetry
  log()
  metric()
  trace()
  event()
```

Services shall depend upon the abstraction rather than directly depending upon the underlying telemetry infrastructure.

This permits the telemetry implementation to change without requiring changes to domain logic.

7. REQUEST CORRELATION

Every externally initiated request shall receive a unique:

`request_id`

The identifier shall propagate through all synchronous and asynchronous operations derived from the request.

Conceptually:

```
REQUEST
  ↓
  request_id
  ↓
  ARIA COMMAND
  ↓
  job_id
  ↓
  worker
  ↓
  geometry operation
  ↓
  validation
  ↓
  drawing generation
```

↓
document package

The complete operational chain must remain discoverable.

8. TRACE IDENTIFIERS

Distributed operations shall additionally use:

trace_id
span_id

A trace represents the complete operation.

A span represents an individual operation within that trace.

Example:

```
TRACE
Create Drawing Package
├── API Request
├── Authorization
├── Revision Retrieval
├── Validation
├── Drawing Generation
├── Geometry Retrieval
├── Sheet Composition
├── PDF Generation
├── Artifact Storage
└── Package Finalization
```

9. PROJECT CORRELATION

Where applicable, telemetry shall include:

organization_id
project_id
revision_id
model_version_id
job_id
command_id
artifact_id
validation_run_id

This permits system operators to trace a technical failure back to the exact OneDraft project state affected.

10. ACTOR CORRELATION

Operational telemetry may include:

```
user_id
service_id
worker_id
api_client_id
```

The system must distinguish:

```
HUMAN ACTOR
AI ACTOR
SYSTEM ACTOR
WORKER ACTOR
EXTERNAL SERVICE
```

This distinction is essential when reconstructing system activity.

11. STRUCTURED LOGGING

All application logs shall be structured.

Plain-text diagnostic output should not be relied upon for production analysis.

Conceptually:

```
{
  "timestamp": "ISO-8601",
  "level": "ERROR",
  "service": "geometry-worker",
  "event": "GEOMETRY_GENERATION_FAILED",
  "project_id": "uuid",
  "model_version_id": "uuid",
  "job_id": "uuid",
  "trace_id": "uuid",
  "error_code": "GEOMETRY_INVALID",
  "message": "Generated geometry failed validation."
}
```

Logs shall be machine searchable.

12. LOG LEVELS

Initial log levels:

```
DEBUG
INFO
WARN
ERROR
```

FATAL

Production systems should avoid unrestricted DEBUG logging.

Sensitive information must not be written to logs.

13. LOG CATEGORIES

OneDraft logs shall distinguish:

APPLICATION
DOMAIN
SECURITY
DATABASE
INTEGRATION
WORKER
GEOMETRY
COMPLIANCE
VALIDATION
DOCUMENTATION
ARTIFACT
PERFORMANCE
SYSTEM

This allows targeted operational investigation.

14. METRIC CATEGORIES

The platform shall expose metrics for:

REQUESTS
LATENCY
ERRORS
THROUGHPUT
QUEUE DEPTH
WORKER UTILIZATION
DATABASE PERFORMANCE
STORAGE
GEOMETRY COMPUTATION
VALIDATION
DRAWING GENERATION
DOCUMENT GENERATION
AI OPERATIONS
AUTHENTICATION
EVENT PROCESSING

15. REQUEST METRICS

The API shall measure:

request_count
request_duration
request_errors
request_rate
request_status_distribution

Metrics shall be partitionable by:

endpoint
method
status
service

High-cardinality identifiers such as project UUIDs should not be used indiscriminately as metric labels.

16. SERVICE HEALTH

Every major OneDraft service shall expose a health interface.

Conceptually:

/health

The health system shall distinguish:

LIVE
READY
DEGRADED
FAILED

17. LIVENESS

Liveness determines whether the process itself is functioning.

Example:

GET /health/live

A liveness failure indicates that the process may need restart or replacement.

18. READINESS

Readiness determines whether the service can accept work.

Example:

GET /health/ready

Readiness may depend upon:

database connectivity
required service availability
configuration validity
queue connectivity
required runtime dependencies

19. DEPENDENCY HEALTH

Services shall be capable of reporting dependency state.

Example:

PostgreSQL	READY
Event Bus	READY
Object Storage	READY
Job Queue	READY
Telemetry	READY

A service may remain alive while being unavailable for normal work.

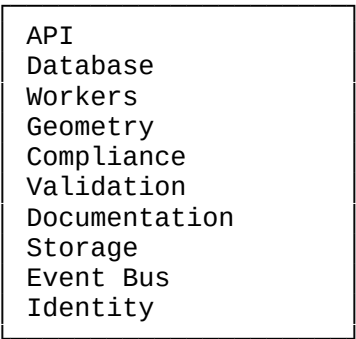
20. SYSTEM HEALTH MODEL

OneDraft shall calculate an aggregate platform health state.

Conceptually:

SYSTEM HEALTH

↓



The aggregate state shall be derived from subsystem health rather than manually entered.

21. DATABASE TELEMETRY

PostgreSQL shall be monitored for:

- connection utilization
- query latency
- transaction duration
- deadlocks
- lock contention
- failed transactions
- connection failures
- replication status
- storage utilization

Long-running transactions shall be identifiable.

22. DATABASE PERFORMANCE

The observability system shall identify:

- slow queries
- high-frequency queries
- failed queries
- deadlocks
- connection pool exhaustion
- index performance anomalies

Application-level tracing should correlate slow database operations with their originating request.

23. JOB TELEMETRY

Every asynchronous job shall generate telemetry.

Required identifiers:

- job_id
- job_type
- project_id
- revision_id
- model_version_id
- worker_id

Required measurements:

- queue_time

start_time
execution_time
completion_time
retry_count
failure_count

24. QUEUE TELEMETRY

The job system shall expose:

queue_depth
oldest_job_age
jobs_per_minute
failure_rate
retry_rate
worker_count
worker_utilization

A rapidly increasing queue depth shall trigger operational attention.

25. WORKER HEALTH

Workers shall periodically report:

worker_id
worker_type
status
current_job
uptime
heartbeat
resource utilization

Worker states:

STARTING
READY
BUSY
DRAINING
UNHEALTHY
STOPPED

26. GEOMETRY TELEMETRY

The geometry subsystem shall measure:

geometry_generation_duration
geometry_validation_duration

geometry_failure_rate
geometry_job_count
geometry_cache_hit_rate
geometry_version_mismatch_count

Geometry failures must be correlated with the corresponding model version.

27. COMPLIANCE TELEMETRY

The compliance engine shall measure:

code_source_retrieval
manifest_generation
rule_evaluation
applicability_evaluation
compliance_check_duration
compliance_failure_count

Code-source retrieval failures shall be distinguished from actual project compliance failures.

28. VALIDATION TELEMETRY

Validation operations shall measure:

validation_runs
validation_duration
rules_executed
results_generated
critical_failures
warning_count
validation_queue_time

The telemetry must identify the:

revision
model version
code manifest

used during validation.

29. DOCUMENTATION TELEMETRY

Drawing generation shall measure:

view_generation
drawing_generation
sheet_composition

annotation_generation
schedule_generation
rendering
export

Document generation shall additionally measure:

package_generation
file_generation
artifact_hashing
artifact_storage

30. ARIA TELEMETRY

Aria operations require specialized telemetry.

The system shall measure:

aria_request_count
aria_command_count
command_generation_duration
command_validation_duration
command_execution_duration
command_failure_rate
command_rejection_rate

Where permitted by the platform's privacy and data-handling rules, model/provider metadata may include:

provider
model_identifier
request_tokens
response_tokens
latency
estimated_cost

The platform should not depend on any single AI provider's telemetry format.

31. AI COMMAND TRACE

An Aria request shall be traceable through:

USER REQUEST
↓
ARIA ORCHESTRATION
↓
CONTEXT ASSEMBLY
↓
MODEL PROVIDER
↓

COMMAND PROPOSAL
↓
COMMAND VALIDATION
↓
DOMAIN EXECUTION
↓
REVISION
↓
VALIDATION

The observability system records the execution path without making the model provider itself part of the OneDraft domain contract.

32. EVENT BUS TELEMETRY

Event processing shall expose:

events_published
events_consumed
consumer_lag
processing_duration
processing_failures
dead_letter_count
retry_count

Every event-processing operation should be correlated with:

event_id
event_type
producer
consumer

33. ARTIFACT TELEMETRY

Artifact operations shall measure:

upload_duration
download_duration
storage_failures
hash_generation_duration
artifact_size
package_generation_duration

The artifact identifier shall permit correlation with the originating document package.

34. SECURITY TELEMETRY

Security telemetry shall include:

- authentication_success
- authentication_failure
- authorization_failure
- token_validation_failure
- rate_limit_trigger
- suspicious_request
- account_lockout

Security events shall be routed through the security monitoring boundary established by Stack 29.0.

35. AUDIT DISTINCTION

Operational logs shall not replace audit records.

For example:

LOG:
API request completed in 412 ms.

AUDIT EVENT:
User created Revision 12.

DOMAIN EVENT:
REVISION_CREATED.

These are three different records serving three different purposes.

36. ALERTING

The observability system shall support alerts for conditions such as:

- service unavailable
- database unavailable
- queue saturation
- worker failure
- high error rate
- high latency
- storage failure
- repeated geometry failures
- validation subsystem failure
- document generation failure
- authentication anomaly
- event consumer failure

Alerts should be based on defined thresholds or anomaly policies rather than arbitrary manual observation.

37. ALERT SEVERITY

Initial severity levels:

INFO
WARNING
ERROR
CRITICAL

Critical alerts represent conditions requiring immediate operational attention.

38. SERVICE-LEVEL OBJECTIVES

Production OneDraft deployments shall eventually define SLOs for:

API availability
API latency
job completion
document generation
validation availability
database availability
artifact availability
event processing latency

SLOs shall be measurable from telemetry.

39. ERROR BUDGET

Once production SLOs are established, each major service may maintain an error budget.

The error budget informs:

release decisions
capacity planning
incident response
reliability investment

40. DASHBOARD ARCHITECTURE

The observability platform shall provide dashboards for:

SYSTEM OVERVIEW
API
DATABASE
WORKERS
QUEUES
GEOMETRY
COMPLIANCE
VALIDATION
DOCUMENTATION
ARIA
EVENT BUS
STORAGE
SECURITY

41. SYSTEM OVERVIEW DASHBOARD

The primary dashboard shall display:

overall health
active jobs
queue depth
request rate
error rate
API latency
worker health
database health
storage health
event processing state

42. PROJECT OPERATIONAL VIEW

Authorized operators shall be able to inspect the operational state of a project.

The view may include:

project_id
current revision
current model version
active jobs
failed jobs
validation state
drawing generation state
document package state
recent operational errors

This view must respect project authorization.

43. TRACE INVESTIGATION

Operators shall be able to retrieve a trace using:

```
trace_id  
request_id  
job_id  
command_id
```

The trace should expose the complete execution chain.

Example:

```
API  
↓  
ARIA  
↓  
COMMAND  
↓  
MODEL  
↓  
GEOMETRY  
↓  
VALIDATION  
↓  
DRAWING  
↓  
DOCUMENT
```

44. ERROR IDENTIFICATION

Errors shall use stable machine-readable codes.

Examples:

```
DATABASE_UNAVAILABLE  
AUTHORIZATION_FAILED  
CONSTRAINT_CONFLICT  
GEOMETRY_INVALID  
GEOMETRY_VERSION_MISMATCH  
VALIDATION_FAILED  
DOCUMENT_GENERATION_FAILED  
ARTIFACT_STORAGE_FAILED  
EVENT_PROCESSING_FAILED  
AI_COMMAND_REJECTED
```

Error codes shall not depend upon human-readable error messages.

45. ERROR CONTEXT

An error record should contain sufficient context to reproduce the failure.

Where applicable:

- error_code
- service
- operation
- request_id
- trace_id
- project_id
- revision_id
- model_version_id
- job_id
- command_id
- timestamp

Sensitive payload data must be excluded or redacted.

46. RETRY OBSERVABILITY

Retries shall be visible.

Every retryable operation shall expose:

- attempt_number
- maximum_attempts
- retry_reason
- retry_delay

This prevents repeated retries from appearing as unrelated failures.

47. DEAD-LETTER OBSERVABILITY

Failed asynchronous messages that exceed retry policy shall enter the dead-letter mechanism.

Telemetry shall record:

- dead_letter_id
- event_id
- job_id
- failure_reason
- attempt_count
- timestamp

Dead-letter growth shall trigger operational visibility.

48. RESOURCE TELEMETRY

The infrastructure layer shall expose:

- CPU
- memory
- disk
- network
- container health
- worker utilization
- database resources
- storage utilization

Resource telemetry shall support capacity planning.

49. COST TELEMETRY

Where practical, OneDraft shall measure infrastructure and AI consumption.

Potential metrics:

- compute_seconds
- storage_bytes
- database_usage
- network_usage
- AI_requests
- AI_token_usage
- estimated_AI_cost

Cost telemetry must remain separate from customer billing records.

50. DATA PRIVACY

Telemetry shall follow the same security principles as the application.

The observability subsystem must not become an uncontrolled copy of project data.

Telemetry shall minimize:

- personal information
- project content
- customer secrets
- authentication credentials
- AI prompts containing sensitive information
- file contents

Only information required for operational diagnosis should be retained.

51. SECRET REDACTION

The telemetry pipeline shall automatically prevent collection of:

- passwords
- access tokens
- refresh tokens
- API keys
- private keys
- session secrets
- database credentials

Sensitive fields shall be redacted before telemetry leaves the application boundary.

52. RETENTION

Operational telemetry shall have defined retention periods.

The retention policy shall distinguish:

- high-volume metrics
- application logs
- security logs
- traces
- critical incident records

Operational telemetry retention shall not override the separate project-data retention and provenance policy established in later Stack 36.0.

53. TELEMETRY STORAGE

The observability storage system shall be independent of the primary project database where practical.

The architecture becomes:

```
POSTGRESQL
  ↓
PROJECT SYSTEM OF RECORD

OBSERVABILITY STORAGE
  ↓
OPERATIONAL SYSTEM OF RECORD
```

Neither system should be required to store the other's entire data model.

54. FAILURE ISOLATION

A failure in telemetry collection must not normally cause the OneDraft application itself to fail.

Telemetry shall therefore use:

- buffering
- batching
- timeouts
- backpressure
- local fallback

where appropriate.

The application remains operational if the telemetry backend becomes temporarily unavailable.

55. TELEMETRY BACKPRESSURE

Telemetry systems shall protect application workloads from excessive telemetry volume.

The system may:

- sample traces
- aggregate metrics
- batch logs
- drop low-priority diagnostic events

Critical operational and security events shall not be discarded merely because telemetry volume is high.

56. TRACE SAMPLING

Trace sampling may be used at scale.

Sampling policies may distinguish:

- successful routine operations
- slow operations
- failed operations
- security events
- critical jobs

Failures and anomalous operations should receive higher tracing priority.

57. DIAGNOSTIC SNAPSHOTS

For significant failures, the system should be capable of producing a diagnostic snapshot containing:

request context
service versions
configuration version
trace identifiers
job state
dependency health
relevant metrics
error records

The snapshot must not contain secrets.

58. VERSION CORRELATION

Every production service shall expose its software version.

Telemetry should correlate:

application_version
database_schema_version
api_version
worker_version
geometry_engine_version
compliance_engine_version
documentation_engine_version

This permits failures to be associated with specific software releases.

59. MODEL VERSION CORRELATION

Because OneDraft is a computational modeling platform, operational telemetry shall distinguish:

software version
model version
project revision
code manifest
document package

These are separate version dimensions.

60. REPRODUCIBILITY

A diagnostic record should provide enough metadata to determine:

which code executed
which project state was active
which model version was used
which regulatory manifest was used

which job executed
which services participated

This supports reproducibility without requiring unrestricted duplication of project data.

61. INCIDENT IDENTIFICATION

An operational incident shall receive:

incident_id
severity
detected_at
affected_service
affected_scope
status
owner
resolution

Incident records are operational records and are not substitutes for domain audit events.

62. INCIDENT LIFECYCLE

Initial incident states:

DETECTED
ACKNOWLEDGED
INVESTIGATING
MITIGATING
RESOLVED
CLOSED

63. INCIDENT CORRELATION

An incident may reference:

request_ids
trace_ids
job_ids
project_ids
service_versions
deployment_versions
alerts

This creates an operational chain from detection to resolution.

64. MAINTENANCE MODE

OneDraft shall support controlled maintenance states.

Conceptually:

NORMAL
DEGRADED
MAINTENANCE
READ_ONLY
EMERGENCY

Maintenance mode shall prevent unsafe operations while allowing authorized diagnostic access.

65. DEGRADED OPERATION

Where possible, the platform should continue operating in degraded mode.

For example:

DOCUMENTATION AVAILABLE
GEOMETRY WORKER DEGRADED

does not necessarily require the entire platform to become unavailable.

Subsystem dependencies should therefore define their own failure domains.

66. CIRCUIT BREAKERS

External and internal service calls that repeatedly fail may use circuit-breaker behavior.

Conceptually:

CLOSED
↓
FAILURE THRESHOLD
↓
OPEN
↓
RECOVERY TEST
↓
HALF-OPEN
↓
CLOSED

This prevents cascading failures.

67. TIMEOUTS

Every networked service operation shall have an explicit timeout policy.

Unbounded waits are prohibited.

Timeout telemetry shall identify:

```
calling_service
target_service
operation
duration
timeout_threshold
```

68. DEPENDENCY MAP

The observability system shall maintain a machine-readable representation of service dependencies.

Example:

```
API
├── Identity
├── Project Service
├── PostgreSQL
└── Event Bus

Runtime Orchestrator
├── Project Service
├── Job System
└── Event Bus

Documentation Engine
├── Model Service
├── Geometry Engine
├── Validation Engine
└── Artifact Storage
```

This dependency graph assists failure analysis.

69. SERVICE REGISTRY

Each service should identify itself with:

```
service_name
service_version
environment
instance_id
region
deployment_id
```

This metadata becomes part of telemetry.

70. ENVIRONMENT IDENTIFICATION

Telemetry shall distinguish:

LOCAL
DEVELOPMENT
TEST
STAGING
PRODUCTION

Production telemetry must never be confused with development telemetry.

71. HEALTH CHECK AUTOMATION

Health checks shall be continuously evaluated.

The platform should automatically detect:

service failure
worker failure
queue saturation
database degradation
storage failure
event consumer lag

72. SELF-DIAGNOSTIC INTERFACE

OneDraft should eventually expose an authorized diagnostic endpoint.

Conceptually:

GET /api/v1/system/health

The response may include:

overall_status
services
dependencies
queues
workers
database
storage
version
timestamp

Sensitive infrastructure information must not be exposed to ordinary users.

73. ADMINISTRATIVE VISIBILITY

Operational dashboards shall require administrative authorization.

Project-level operational visibility shall require project authorization.

The observability system therefore follows the same principle:

VISIBILITY = AUTHORIZATION

74. USER-FACING STATUS

The internal observability system shall provide controlled status information to the product interface.

Users may see:

Generating drawings...
Validation running...
Document package preparing...
Completed

They should not ordinarily see:

internal hostnames
database credentials
worker topology
private infrastructure identifiers

75. JOB PROGRESS

Long-running operations should publish progress.

Example:

QUEUED	0%
RUNNING	20%
GEOMETRY	40%
VALIDATION	60%
DOCUMENT	80%
FINALIZING	95%
COMPLETED	100%

Progress should represent meaningful workflow stages rather than arbitrary numerical increments.

76. OBSERVABILITY API

The internal API shall eventually expose authorized operational endpoints for:

system health
service health
job status
incident status
operational metrics
trace lookup

These interfaces remain separate from the core project modeling API where appropriate.

77. EVENT-DRIVEN HEALTH

Subsystem health changes may generate operational events.

Examples:

SERVICE_DEGRADED
SERVICE_RECOVERED
WORKER_FAILED
QUEUE_SATURATED
DATABASE_DEGRADED
STORAGE_UNAVAILABLE

These events are operational and must not be confused with project-domain events.

78. RECOVERY DETECTION

Recovery shall be observable.

For every significant incident:

failure detected
↓
mitigation
↓
service recovery
↓
health confirmation
↓
incident resolution

The system should not automatically mark a failure resolved merely because an error rate temporarily decreases.

79. OPERATIONAL AUDITABILITY

The observability system must allow authorized personnel to determine:

- when an operation occurred
- which service executed it
- which version executed it
- which job performed it
- whether it succeeded
- whether it failed
- what dependencies were involved

This establishes operational accountability.

80. SYSTEM DIAGNOSTIC CHAIN

A complete diagnostic chain becomes:

```
USER REQUEST
↓
REQUEST ID
↓
TRACE
↓
SERVICE
↓
COMMAND / JOB
↓
MODEL VERSION
↓
REVISION
↓
DEPENDENCIES
↓
RESULT
↓
ARTIFACT
↓
OPERATIONAL TELEMETRY
```

81. OBSERVABILITY DATA MODEL

The initial operational model should conceptually contain:

- TelemetryRecord
- Metric
- LogRecord
- Trace
- Span
- HealthCheck

ServiceInstance
Alert
Incident
DiagnosticSnapshot

These records should remain operationally distinct from project-domain entities.

82. MINIMUM TELEMETRY CONTRACT

Every major service must emit at minimum:

service_name
service_version
environment
timestamp
request_id
trace_id
operation
status
duration
error_code

Where applicable:

project_id
revision_id
model_version_id
job_id
command_id

83. SERVICE IMPLEMENTATION CONTRACT

Every new OneDraft service introduced after Stack 30.0 shall implement:

health check
structured logging
request correlation
distributed tracing
service metrics
error classification
version reporting

A service is not operationally complete until these interfaces exist.

84. TESTING REQUIREMENT

The observability subsystem shall itself be tested.

Tests shall verify:

- request correlation
- trace propagation
- metric emission
- log formatting
- health reporting
- failure reporting
- alert triggering
- telemetry redaction

85. FAILURE TESTING

The system shall deliberately test:

- database unavailable
- event bus unavailable
- worker unavailable
- storage unavailable
- geometry failure
- validation failure
- document generation failure
- AI provider timeout
- AI provider failure
- network timeout

The expected operational response must be observable.

86. SECURITY TESTING

Telemetry testing shall verify that:

- credentials are never logged
- tokens are never logged
- private keys are never logged
- unauthorized users cannot access diagnostics
- project data is not exposed through public health endpoints

87. PERFORMANCE REQUIREMENT

Telemetry must have bounded application overhead.

The observability subsystem shall be designed so that normal instrumentation does not materially degrade:

- API latency
- job throughput
- geometry performance
- database performance
- document generation

88. SCALABILITY

Telemetry infrastructure must scale independently from application workloads.

As OneDraft grows:

- projects
- users
- commands
- jobs
- drawings
- documents
- events

the observability layer must scale without requiring proportional changes to the domain model.

89. DEVELOPMENT REQUIREMENT

Local development shall provide a lightweight observability environment.

Developers must be able to inspect:

- logs
- metrics
- traces
- job execution
- service health

without requiring production infrastructure.

90. STAGING REQUIREMENT

Staging shall reproduce the production telemetry architecture closely enough to validate:

- instrumentation
- alerts
- dashboards

trace propagation
failure handling
capacity behaviour

before production deployment.

91. PRODUCTION REQUIREMENT

Production shall provide:

centralized logs
metrics
distributed tracing
health monitoring
alerting
incident tracking
operational dashboards

with controlled administrative access.

92. OBSERVABILITY DEPENDENCY BOUNDARY

The observability system shall not become a hard dependency of the computational project model.

The correct dependency is:

PROJECT SERVICES
↓
TELEMETRY INTERFACE
↓
OBSERVABILITY INFRASTRUCTURE

not:

PROJECT SERVICES
↓
OBSERVABILITY DATABASE
↓
PROJECT EXECUTION

93. SYSTEM OF RECORD DISTINCTION

OneDraft now recognizes three distinct operational information classes:

DOMAIN STATE

The computational project model.

DOMAIN HISTORY

Revisions, events, acceptance and provenance.

OPERATIONAL STATE

Logs, metrics, traces, health, alerts and incidents.

These systems may correlate with one another but must not be conflated.

94. OBSERVABILITY AND RECONSTRUCTION

Given:

```
project_id  
revision_id  
model_version_id  
job_id  
trace_id
```

authorized operators should be able to reconstruct the operational path associated with a project operation.

This is complementary to the project reconstruction requirement established by the earlier persistence architecture.

95. FIRST OPERATIONAL INTEGRATION TEST

The first complete observability integration test shall execute:

1. Start OneDraft services
2. Verify service health
3. Create organization
4. Create user
5. Create project
6. Create model

7. Submit Aria command
8. Generate geometry job
9. Run validation
10. Generate drawing
11. Generate document package
12. Store artifact
13. Publish event
14. Complete job
15. Retrieve request trace
16. Retrieve job telemetry
17. Verify project correlation
18. Verify model-version correlation
19. Verify error/metric records
20. Verify sensitive data redaction
21. Verify final system health

The test succeeds only when the entire execution chain is observable.

96. INITIAL OBSERVABILITY ARTIFACTS

The implementation package shall ultimately contain:

```
telemetry/  
  logging.py  
  metrics.py  
  tracing.py  
  health.py  
  alerts.py  
  incidents.py
```

Service integration:

```
packages/  
  observability/
```

Infrastructure:

```
infra/  
  observability/
```

Documentation:

/docs/observability/

97. CONFIGURATION

Observability configuration shall support:

telemetry_enabled
log_level
metric_collection
trace_sampling
export_endpoint
environment
retention_policy
alert_configuration

Configuration secrets shall remain outside source control.

98. VERSIONING

The observability contract shall be versioned independently where necessary.

Changes affecting application telemetry interfaces shall be backward compatible whenever practical.

Breaking changes require an explicit version transition.

99. MVP SUCCESS CRITERIA

Stack 30.0 is successful when OneDraft can:

Determine service health

Correlate requests

Trace distributed operations

Collect structured logs

Collect application metrics

Observe asynchronous jobs

Observe workers

Observe geometry operations

Observe validation
Observe documentation generation
Observe artifact operations
Observe event processing
Observe Aria command execution
Detect failures
Generate alerts
Protect telemetry from secrets
Correlate telemetry with project state
Identify software versions
Investigate an operational incident
Determine whether the platform has recovered

100. FINAL ARCHITECTURAL DECISION

OneDraft observability shall not be treated as an afterthought or debugging utility.

It is a permanent operational subsystem of the platform.

The architecture therefore becomes:

COMPUTATIONAL PROJECT MODEL
+
ARIA INTELLIGENCE
+
GEOMETRY ENGINE
+
REGULATORY ENGINE
+
VALIDATION ENGINE
+
DOCUMENT ENGINE
+
VERSION CONTROL
+
CUSTOMER REVIEW
+
RUNTIME ORCHESTRATION
+
JOB / WORKER EXECUTION
+
ARTIFACT STORAGE
+
EVENT BUS

+
IDENTITY / SECURITY
+
OBSERVABILITY

Observability provides the operational visibility necessary to operate every preceding subsystem as a coherent production system.

101. STACK 30.0 IMPLEMENTATION BOUNDARY

The implementation boundary established by this specification is:

APPLICATION
↓
TELEMETRY INTERFACE
↓
LOGGING
METRICS
TRACING
HEALTH
ALERTING
INCIDENTS
↓
OBSERVABILITY INFRASTRUCTURE

The project domain remains independent of the telemetry implementation.

102. NEXT STACK

With Stack 30.0 established, the architecture proceeds to:

31.0 — Testing, Quality Assurance & Verification System

The next subsystem shall define the automated mechanism by which OneDraft verifies that its project model, geometry, compliance, validation, documentation, runtime, workers, storage, event bus, security controls and observability infrastructure operate correctly as individual components and as one integrated platform.

103. SPECIFICATION STATUS

ONEDRAFT OBSERVABILITY, TELEMETRY & SYSTEM HEALTH v30.0

STATUS: ESTABLISHED

The OneDraft architecture has now advanced from:

DOMAIN ARCHITECTURE

through:

EXECUTABLE APPLICATION ARCHITECTURE

and into:

OPERATIONAL SYSTEM ARCHITECTURE